

MusicDB

Reporte de diseño y desarrollo

Jennifer Jade Herrera Cuamatla

Junio 2026

Descripción del proyecto

El proyecto fue desarrollado en Java, usando JavaFX como herramienta gráfica, SQLite como base de datos y Maven como administrador de proyecto. El despliegue se realiza con Docker con soporte de X11 forwarding, lo que permite ejecutar la interfaz gráfica dentro de un contenedor en cualquier máquina Linux con pantalla.

La elección de Java como lenguaje principal responde a que es el lenguaje con mayor soporte dentro del curso y el que mejor se integra con las herramientas requeridas. JavaFX fue preferido sobre Gtk y Qt porque permite definir las vistas en archivos FXML separados del código Java, lo que facilita mantener la separación entre vista y controlador que exige el patrón MVC. SQLite fue elegido sobre un servidor de base de datos porque el proyecto no requiere acceso concurrente desde múltiples procesos y un archivo local es suficiente, además de simplificar considerablemente la dockerización.

Componentes del sistema

El sistema se divide en cuatro componentes con responsabilidades bien delimitadas.

Base de datos

La base de datos es la fuente de verdad del sistema. Almacena y organiza la información musical de forma estructurada y consultable. No tiene conocimiento de los archivos MP3, ni de la interfaz gráfica; simplemente guarda datos y responde consultas. El esquema fue proporcionado por el curso y refleja una estructura normalizada en la que los intérpretes, álbumes y canciones son entidades independientes relacionadas entre sí.

Componente DAO

La capa DAO (Data Access Object) actúa como traductor entre los objetos Java y las tablas de SQLite. Sin esta capa, el código SQL quedaría disperso por toda la aplicación, haciendo difícil cualquier modificación al esquema y complicando las pruebas. Cada clase DAO agrupa todas las operaciones relacionadas con una entidad en un solo lugar.

Se definió una interfaz genérica `DAO<T, ID>` que establece el contrato básico de operaciones CRUD: `insertar`, `buscarPorId`, `buscarTodos`, `actualizar` y `eliminar`. Cada DAO concreto implementa esta interfaz y agrega los métodos de búsqueda propios de su entidad.

Las clases DAO del proyecto son: `RolaDAO`, `PerformerDAO`, `AlbumDAO`, `PersonDAO`, `GroupDAO` e `InGroupDAO`. Esta última no extiende la interfaz genérica porque su clave primaria es compuesta (`id_person`, `id_group`), lo que no encaja en el patrón de ID simple.

El minero

El minero es el componente encargado de poblar la base de datos. Recibe una ruta de directorio, lo recorre recursivamente y procesa cada archivo `.mp3` encontrado. Para cada archivo, lee sus etiquetas ID3v2.4 usando la librería `JAudioTagger` y construye los objetos correspondientes en la base de datos.

El flujo de procesamiento de cada archivo es:

1. Leer las etiquetas ID3: título, artista, álbum, género, año y número de pista.
2. Verificar si esa rola ya existe en la BD mediante su `path`. Si existe, se ignora el archivo.
3. Insertar un nuevo `Performer` de tipo `Unknown` con el nombre del artista leído.
4. Buscar si ya existe un álbum con el mismo nombre. Si existe, reutilizar su `id_album`. Si no, insertarlo.
5. Insertar la rola con los IDs obtenidos.

Las etiquetas ID3 solo proveen un string con el nombre del artista, sin indicar si es una persona, un grupo u otro tipo de entidad. En lugar de intentar inferirlo heurísticamente o interrumpir el proceso para preguntarle al usuario, se tomó la decisión de insertar todos los performers nuevos como `Unknown`. El usuario puede clasificarlos correctamente desde la interfaz gráfica después de ejecutar el minero.

Interfaz gráfica

La interfaz gráfica fue desarrollada con JavaFX siguiendo el patrón MVC. Las vistas se definen en archivos FXML, los controladores manejan los eventos del usuario y los modelos son los mismos objetos que usa la capa DAO. La comunicación entre la vista y la base de datos siempre pasa por el controlador y los DAOs; nunca directamente.

Compilador de búsquedas

El cuarto componente es un compilador de búsquedas que permite al usuario expresar consultas en un lenguaje sencillo en lugar de escribir SQL directamente. La gramática definida para este lenguaje está inspirada en la lógica proposicional clásica, restringida a negación, conjunción y disyunción.

La sintaxis es la siguiente:

campo:valor	búsqueda por campo con valor
,	conjunción (AND)
/	disyunción (OR)
!	negación
()	agrupación
=, >, <, >=, <=	comparaciones numéricas para anio y track

Ejemplos de expresiones válidas:

```
performer:"queen" , genero:"rock"  
genero:!"pop" / anio>=1980  
titulo:"love" , (performer:"beatles" / performer:"stones")  
anio>=1970 , anio<=1979
```

El compilador toma una expresión como esta, la parsea y genera dinámicamente la query SQL correspondiente, que luego se ejecuta a través de RoladaDAO.

Diagrama de clases

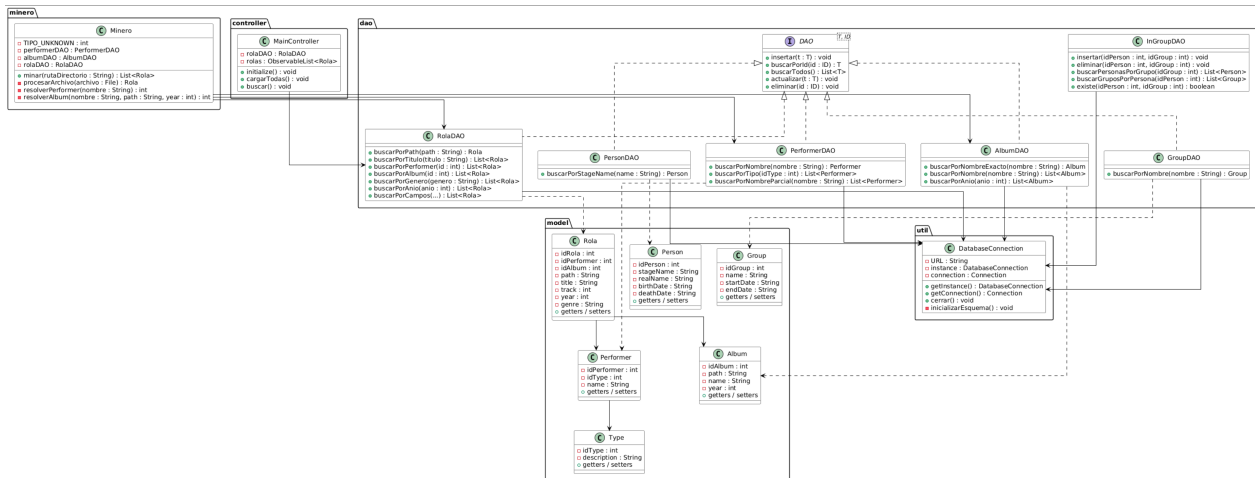


Figura 1: Diagrama de clases de MusicDB

Dockerización

La aplicación se empaqueta en una imagen Docker mediante un proceso de construcción en dos etapas. La primera etapa usa una imagen con JDK completo y Maven para compilar el proyecto y producir el JAR ejecutable. La segunda etapa usa una imagen más ligera con solo el JRE, copia el JAR y las dependencias, e instala las librerías de sistema necesarias para JavaFX en Linux.

Para que la interfaz gráfica funcione dentro del contenedor, se utiliza X11 forwarding: el socket `/tmp/.X11-unix` del host se monta como volumen y la variable de entorno `DISPLAY` se pasa al contenedor. La ventana de la aplicación aparece en la pantalla del usuario como si fuera un proceso local.

La base de datos SQLite se persiste en un volumen montado en `/app/data`, de modo que los datos sobreviven el ciclo de vida del contenedor. La ruta de la base de datos se configura mediante la variable de entorno `DB_PATH`, lo que permite usar una ruta local fuera de Docker durante el desarrollo y la ruta del volumen al ejecutar en contenedor.

Dificultades y retos

El desarrollo del proyecto presentó varios obstáculos que requirieron investigación y decisiones de diseño adicionales.

El primero fue previo al desarrollo mismo: la computadora utilizada dejó de conectarse al monitor externo y de tener acceso a internet después de una actualización del sistema. El diagnóstico indicó que el problema era un módulo del driver NVIDIA que falló al instalarse para el kernel nuevo. La solución fue arrancar un kernel anterior desde el menú GRUB, fijar ese kernel como predeterminado y limpiar completamente los paquetes NVIDIA para reinstalarlos desde cero. Este proceso tomó tiempo considerable antes de poder retomar el trabajo.

En cuanto al proyecto en sí, la primera dificultad fue la configuración de JavaFX dentro de Docker. El intento inicial de empaquetar la aplicación con `maven-shade-plugin` producía un JAR que fallaba al iniciar con el mensaje *JavaFX runtime components are missing*. Esto se debe a que JavaFX no puede empaquetarse en un fat JAR de la manera convencional porque sus módulos requieren un tratamiento especial en la JVM. La solución fue cambiar a `maven-dependency-plugin` para copiar las dependencias junto al JAR y pasar los módulos explícitamente al comando `java` mediante `-module-path` y `-add-modules`.

Otra dificultad fue que el paquete `libgl1-mesa-glx`, necesario para el renderizado de JavaFX en Linux, fue eliminado en Ubuntu 24.04 y reemplazado por `libgl1`. Esto causó un error durante la construcción de la imagen Docker que no era evidente a primera vista.

En cuanto al diseño del minero, la dificultad principal fue decidir cómo manejar el tipo de performer. El esquema distingue entre `Person`, `Group` y `Unknown`, pero las etiquetas ID3 no contienen esa información. Se consideraron varias alternativas antes de optar por insertar

siempre como **Unknown** y delegar la clasificación al usuario desde la interfaz gráfica, lo que simplificó el minero sin sacrificar la integridad del modelo.

Finalmente, la configuración de dos cuentas de GitHub en la misma máquina requirió entender el sistema de llaves SSH múltiples y el archivo de configuración `~/.ssh/config` para asignar cada alias de host a su llave correspondiente, así como el uso de `git config` local para que cada repositorio usara las credenciales correctas.

Conclusiones

El desarrollo de MusicDB permitió integrar en un solo proyecto varias áreas que en el curso se habían trabajado por separado: diseño de bases de datos relacionales, patrones de diseño orientados a objetos, interfaces gráficas y contenerización con Docker.

Una de las lecciones más importantes fue la relevancia de definir las responsabilidades de cada componente antes de escribir código. Las decisiones tomadas durante el análisis, como separar la capa DAO de la lógica de negocio o establecer que la interfaz no modifica archivos físicos, evitaron problemas de diseño que habrían sido costosos de corregir más adelante.

También resultó valioso aprender que Docker no es solo una herramienta de despliegue final, sino que conviene integrarlo desde el inicio del desarrollo. Haberlo configurado desde las primeras etapas permitió detectar problemas de compatibilidad, como el de los módulos JavaFX, en un momento en que aún era fácil corregirlos.